

SYSTEM REQUIREMENT SPECIFICATION

GROUP SUCH

SHOPPING SYSTEM

TABLE OF CONTENT

TABLE OF CONTENT

1. Introduction	1
1.1. Problem Definition	1
1.2. Purpose.....	1
1.3. Scope.....	1
1.4. Definition, Acronyms and Abbreviations	2
1.5. References	3
1.6. Overview	3
2. Overall Description	3
2.1 Product Perspective	3
2.1.1. System Interfaces	4
2.1.2. User Interfaces	4
2.1.3. Hardware Interfaces	4
2.1.4 Software Interfaces	4
2.1.5. Communication Interfaces.....	5
2.1.6. Memory.....	5
2.1.7. Operations.....	5
2.1.8. Site Adaptation Requirements	5
2.2. Product Functions.....	6
2.3. Constraints.....	8
2.4. Assumptions and Dependencies.....	8
3. Specific Requirements.....	8
3.1. Interface Requirements	8
3.2. Functional Requirements	8
3.2.1 Sign Up	8
3.2.2 Sign In.....	9
3.2.3 Sign Out	10
3.2.4 Add Transportation Route.....	11
3.2.5 Delete Transportation Route	12
3.2.6 Request Transportation Route.....	12

TABLE OF CONTENT

3.2.7 Search Transportation Route.....	13
3.2.8 Send Message.....	13
3.2.9 Reply to Message.....	14
3.2.10 Block User.....	15
3.2.11 Rate User	16
3.2.12 Change Language	16
3.3. Non-Functional Requirements	17
3.3.1. Performance Requirements	17
3.3.2. Design Constraints	17
4. Planning	32
4.1. Team Structure.....	32
4.2. Estimation	32
4.3. Process Mode.....	32
5. Conclusion.....	32
6. Supporting Information	33

TABLE OF CONTENT

LIST OF FIGURES

Figure 1. Use Case Activity Diagram.....	7
Figure 2. Class Diagram	18
Figure 3. Visualization of Main Page.....	22
Figure 4. Visualization of Sign In Page.....	22
Figure 5. Visualization of Sign Up Page	23
Figure 6. Visualization of Profile (Owner) Page	24
Figure 7. Visualization of Search Page	25
Figure 8. Visualization of Route Page.....	26
Figure 9. Visualization of Message Box Page.....	26
Figure 10. Visualization of Profile Page.....	27
Figure 11. Visualization of Message Page.....	28
Figure 12. Visualization of My Transportation Page.....	28
Figure 13. Visualization of Add New Transportation Page.....	29
Figure 10. Visualization of Events Page.....	30

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

1. Introduction

1.1. Problem Definition

In the current fast-paced lifestyle, people find it difficult to visit physical clothing stores due to traffic congestion, limited time, and the hassle of finding specific sizes or styles. Traditional shopping consumes significant time and effort.

Additionally, store owners face challenges in managing inventory manually and reaching a broader audience beyond their local geographical location. “**RAWNAQ**” aims to solve these issues by providing a centralized online platform where users can browse, filter, and purchase fashion items (Men & Women) from the comfort of their homes, while providing administrators with digital tools to manage stock and products efficiently.

1.2. Purpose

The aim of this Software Requirements Specification (SRS) document is to provide a complete description of all the features and specifications for the “**RAWNAQ**” system.

It defines the expectations from the project, including the user interface, database interactions, and functional modules (Shopping, Payment, Admin Dashboard).

This document serves as a guide for the development team (Team 5) to ensure all requirements found in the source code (Homepage.py, Admin_Dashboard.py, etc.) are documented and verified.

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

1.3. Scope

The **RAWNAQ** Project is a web-based E-Commerce application developed using **Python** and **Streamlit**.

- **For Users:** It provides a seamless shopping experience allowing registration, product browsing with filters (Category, Price, Name), cart management, and secure checkout simulation (Visa/Cash).
-
- **For Admins:** It provides a secured Dashboard to perform CRUD operations (Create, Read, Update, Delete) on the product inventory.
- The system uses **SQLite** as a database backend to store user data, products, orders, and feedback.

1.4. Definition, Acronyms and Abbreviations

The definitions of the terms, which are used in this SRS document, are shown below:

Terms	Definitions
GUI	Graphical User Interface (Streamlit Web Interface)
SRS	System Requirements Specification.
CRUD	Create, Read, Update, Delete (Admin Operations).
Admin	A user with elevated privileges to manage the system
Guest	An unregistered user who can only view products.
Client	The registered user buying products
DBMS	Database Management System (SQLite).
Streamlit	The Python framework used for frontend and backend logic.

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

1.5. References

2. Python Documentation (v3.x)
3. Streamlit Documentation.
4. SQLite Database Documentation.
5. IEEE STD 830-1998, IEEE Recommended Practice for Software Requirements Specifications.

1.6 . Overview

The rest of this document contains the overall description of the **RAWNAQ** system, including interface properties, product functions, and dependencies. It details specific functional requirements (Use Cases) derived from the project code, data models representing the '**database.db**' structure, and behavioral descriptions of the user journey.

2. Overall Description

2.1 Product Perspective

RAWNAQ is a standalone web application. It integrates a frontend user interface (built with **Streamlit**) with a backend logic layer (team5.py) and a persistent data storage layer (**database.db**).

The system is designed to be modular, separating the Admin Dashboard logic from the Customer interface.

2.2 Product Functions

The system supports the following major functions:

- **User Authentication:** Registration and Login (distinguishing between 'user' and 'admin' roles).
 - **Product Catalog:** Displaying products for Men and Women with details (Price, Size, Stock, Image).
 - **Search & Filter:** Filtering products by category, name text, and price range sliders.
 - **Cart Management:** Adding items, updating quantities, and removing items.
-

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

- **Checkout Process:** Address validation, Payment simulation (Visa validation/Cash on Delivery), and Stock deduction.
- **Inventory Management (Admin):** Uploading images, setting prices/stock, editing details, and deleting products.
- **Profile Management:** Users can update their username/password and view their transaction history.

2.3 User Classes and Characteristics

1. Administrator:

- Has full access to the Admin_Dashboard.py.
- Can manipulate the products table in the database.
- Technical skill: Basic understanding of inventory data.

2. Registered User:

- Can access Homepage.py, Profile, Cart, and Payment.
- Can perform transactions.

3. Guest (Unregistered):

- Can view the Homepage and products but is redirected to Register.py when attempting to add items to the cart.

2.4 Operating Environment

- **OS:** Cross-platform (Windows, Linux, macOS).
- **Language:** Python 3.x.
- **Browser:** Chrome, Firefox, Safari, Edge.
- **Dependencies:** streamlit, pandas, Pillow (PIL), sqlite3.

2.5 Design and Implementation Constraints

- **Session Management:** Streamlit's session state is used to handle user logins and cart persistence (st.session_state).
- **Database:** SQLite is used, which is file-based (database.db), suitable for this scale but requires careful connection handling (implemented in team5.py).
- **UI Customization:** Limited to Streamlit's layout capabilities and custom CSS injection for fonts ("Times New Roman") and background images.

2.6 Assumptions and Dependencies

- It is assumed the host machine has Python and the required libraries installed.
 - The product images folder exists to store uploaded product photos.
 - Users have a stable internet connection to load the web interface.
-

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

3. Specific Requirements

3.1 Interface Requirements

The system shall consist of user-friendly web-based interfaces.

- **Navigation:** A sidebar menu for quick access to Home, Profile, Cart, About Us, and Logout.
- **Visuals:** The system uses a specific color palette (Brown/Beige earth tones) and background images (payment.jpg, logbg.jpg) to reflect the brand identity.
- **Responsiveness:** The layout adapts to column structures (e.g., 4 columns for product grids).

3.2. Functional Requirements

3.2.1. Sign Up

Use Case ID	UC1
Actor(s)	User
Description	Guest
Preconditions	User is not logged in.
Post conditions	User will be able to log in to the system
Precedence	Mandatory
Normal flow of event	1. User selects "Sign Up". 2. User enters Username and Password. 3. System checks if username exists. 4. System creates record in users table. 5. Success message displayed.
Alternative Flow	If username exists, system shows "Username already exists".

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

3.2.2. Sign In

Use Case ID	UC2
Actor(s)	User/Admin
Description	User Log In
Preconditions	The user shall be able to sign in to the system.
Post conditions	User will be able to use the system.
Precedence	Mandatory
Normal flow of event	1. User enters Username and Password. 2. System validates credentials via team5.login_check. 3. If role is 'admin', redirect to Admin_Dashboard.py. 4. If role is 'user', redirect to Homepage.py.

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

3.2.3. Sign Out

Use Case ID	UC3
Actor(s)	User
Description	User Log Out
Preconditions	The user shall be able to log in to the system.
Post conditions	User will be able to leave the system.
Precedence	Not mandatory
Normal flow of event	1. User presses the log out button. 2. User leaves the system. 3. The system's Home page will be loaded.

3.2.4. Browse & Filter Products

Use Case ID	UC4
Actor(s)	User
Description	View products based on criteria.
Preconditions	The user shall be able to sign in to the system.
Post conditions	User shall be retrieve transportation requests from the other users.
Precedence	Mandatory
Normal flow of event	1. User selects Category (Men/Women). 2. User adjusts Price Slider or types Name. 3. System queries products table.

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

	4. Grid displays matching items with Images and Price.
--	--

3.2.5. Add to Cart

Use Case ID	UC5
Actor(s)	User logged in
Description	User shall be able to add to cart.
Preconditions	The user shall be able to sign in to the system.
Post conditions	User cannot add to cart without log in.
Precedence	No mandatory
Normal flow of event	1. User clicks "Add to Cart" on a product. 2. System checks stock availability. 3. System inserts/updates record in cart table via team5.add_to_cart. 4. Toast notification "Added to Cart" appears.
Alternative Flow	If user is not logged in, redirect to Login page.

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

3.2.6. Checkout & Payment

Use Case ID	UC6
Actor(s)	User
Description	User shall be able to pay
Preconditions	The user shall complete order.
Post conditions	User will be able to rate the process.
Precedence	No mandatory
Normal flow of event	1. User reviews cart total. 2. User clicks "Checkout". 3. User selects Payment Method (Visa / Cash). 4. If Visa: User enters Card No (16 digits) and CVV. 5. System validates inputs. 6. System decrements stock from products and clears cart. 7. Success message/Balloons appear.

3.2.7. Admin: add Product

Use Case ID	UC7
Actor(s)	Admin
Description	Add new inventory item.
Preconditions	Role must be 'admin'.
Post conditions	User will be able to add product to the Home page.
Precedence	No mandatory

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

Normal flow of event	1. Admin navigates to "Add New Product". 2. Fills Name, Category, Price, Stock, Size. 3. Uploads Image. 4. System saves image to {product_images} and data to DB.
----------------------	---

3.2.8. Admin: Edit/Delete Product

Use Case ID	UC8
Actor(s)	Admin
Description	Admin shall be able to delete or edit the products.
Preconditions	The role must be Admin.
Post conditions	Admin will be able to delete or edit products.
Precedence	No mandatory
Normal flow of event	1. Admin selects product from list. 2. Updates fields (Price/Stock) or clicks "Delete". 3. System updates/removes row in products table.

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

Figure 1 illustrates the main interactions between the users and the RAWNAQ E-Commerce System. The system involves three primary actors:

- **Guest:** A user who is not logged in. They can only **Browse Products**, **Search**, and **Filter** items by category or price.
- **Registered User (Client):** A user who has logged in. In addition to browsing, they can **Add Items to Cart**, **Manage Profile**, **Update Password**, and perform **Checkout** to complete a purchase.
- **Admin:** A user with special privileges who accesses the Dashboard to **Add Products**, **Edit Product Details**, **Delete Products**, and **Monitor Inventory**.

The diagram below depicts these relationships and the system boundaries

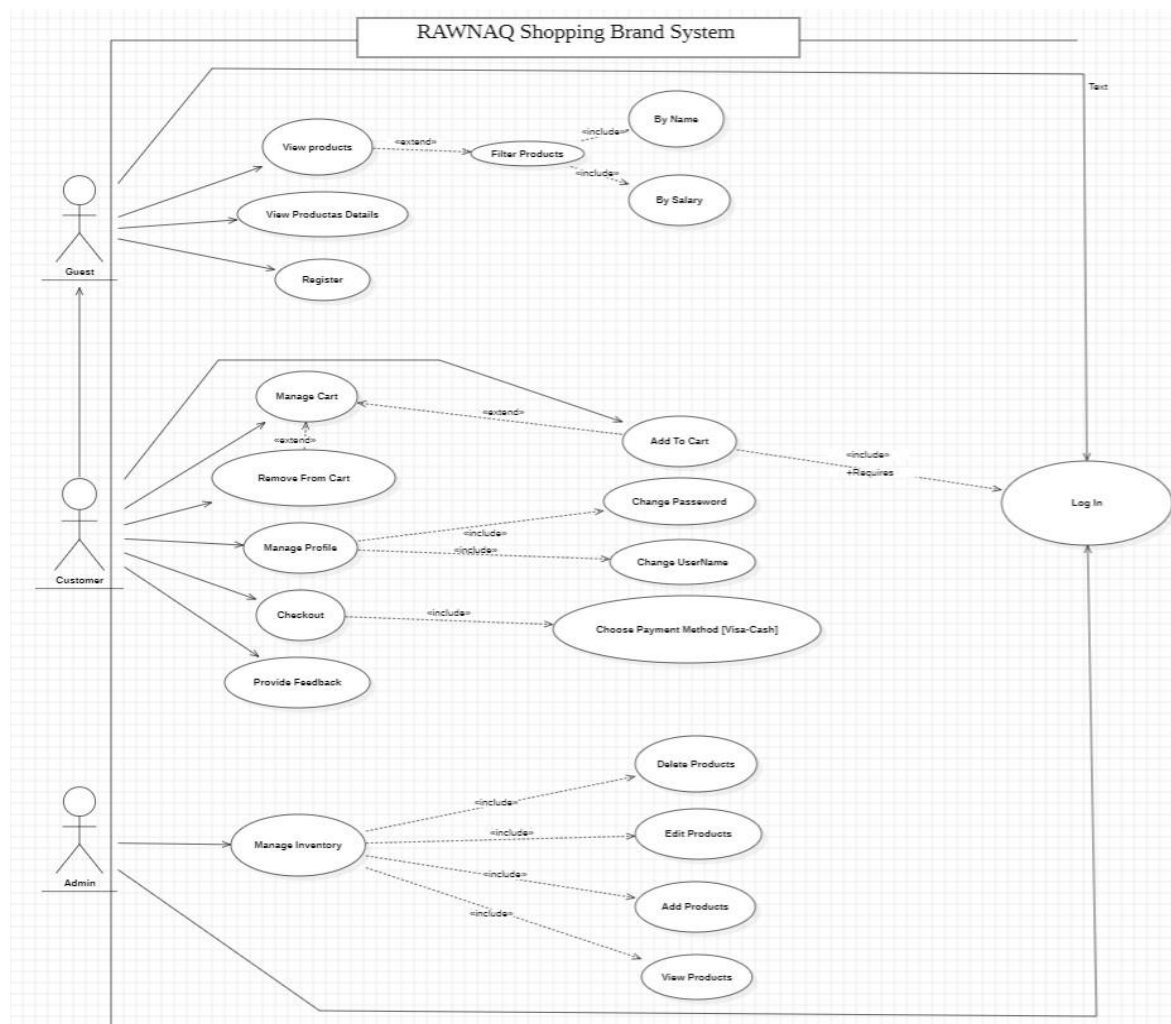


Figure 1. Use Case Diagram

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

3.3. Non-Functional Requirements

3.3.1. Performance Requirements

- The system shall load product images within 2 seconds.
- Database queries (sqlite3) should execute in under 1 second for standard inventory sizes.
- The interface must handle state reruns (st.rerun) smoothly without losing user session data.

3.3.2. Design Constraints

- **Security:** Passwords must be handled securely. The current system stores them in the DB (Note: For future iterations, hashing is recommended).
- **Validation:** Input fields (Visa number, Phone) must be validated for correct length and type (digits only) before submission.
- **Maintainability:** Code is split into modules (team5.py for logic, pages for UI) to ensure easy maintenance.

4. Data Model and Description

4.2. Data Description

4.2.1. Data Objects

The system is built upon a Relational Database schema. The key entities are:

- **Users:** Stores account info and roles.
 - **Products:** Stores inventory details.
 - **Cart:** Links Users to Products with quantities.
 - **Orders/Payment:** Stores transaction history.
-

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

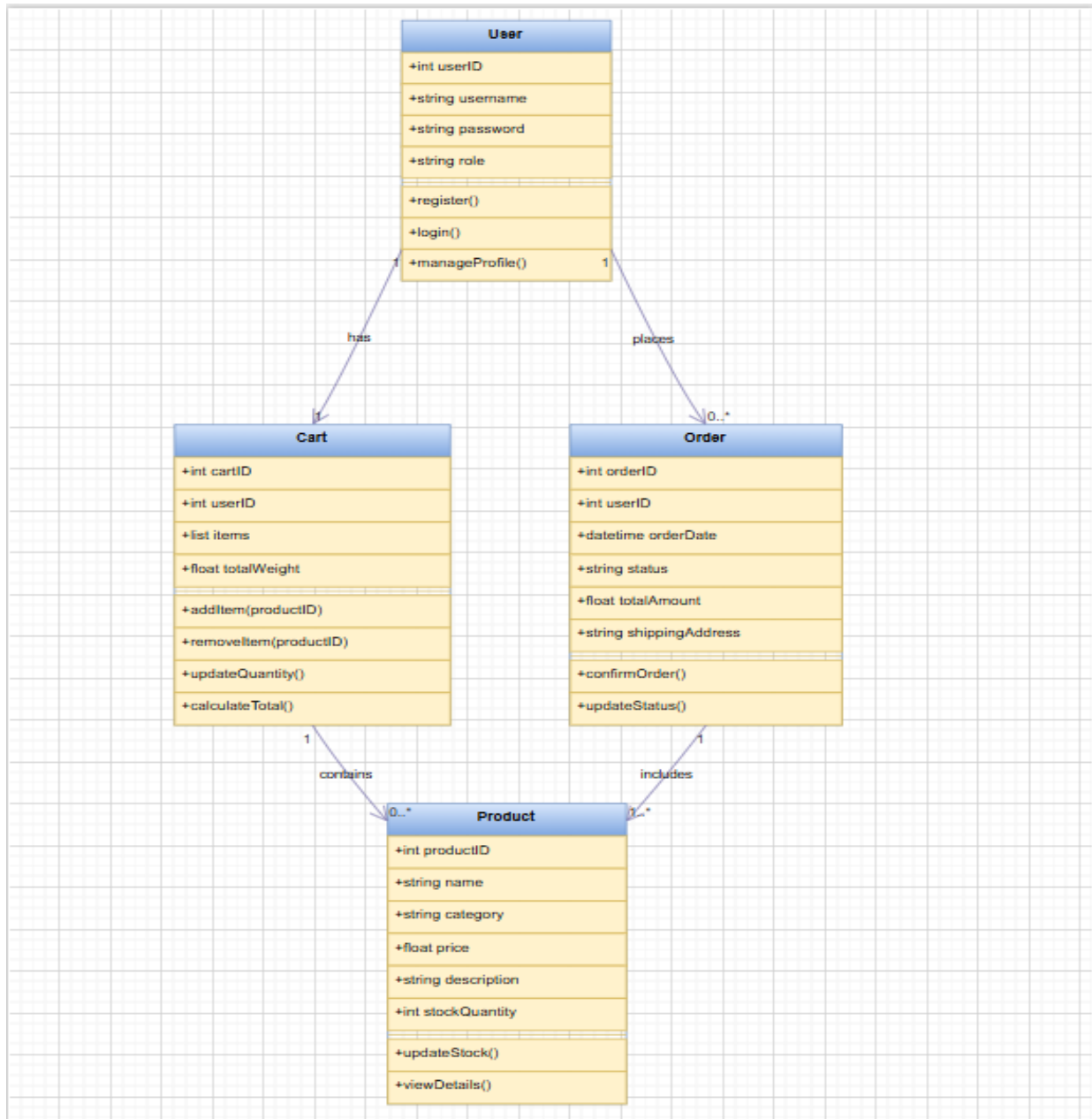


Figure 2. Class Diagram

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

Figure 2 depicts the static structure of the RAWNAQ system. The main classes representing the database entities are:

- **User:** Represents the registered client or admin. It holds attributes like username, password, And role. It is associated with the **Cart** class.
- **Product:** Represents the fashion items available for sale. It contains details such as name, Salary (price), stock, size, and image. Each product belongs to a specific **Category**.
- **Category:** Represents the classification of products (e.g., Men, Women). It has a one-to-many relationship with the **Product** class.
- **Cart:** Acts as a linking entity between **User** and **Product**. It stores the quantity of items a user intends to purchase.
- **Admin:** A specialized type of User (inherits from User) responsible for managing the inventory (CRUD operations).

4.2.2. Data Dictionary

Based on (init_db):

Table: Users | Field | Type | Description | | :--- | :--- | :--- | | id | INTEGER PK | Unique User ID (Auto-increment) | | username | TEXT | Unique login name | | password | TEXT | User password | | role | TEXT | 'admin' or 'user' | | address | TEXT | User shipping address |

Table: Products | Field | Type | Description | | :--- | :--- | :--- | | id | INTEGER PK | Unique Product ID | | category_id | INTEGER | FK ref Categories | | name | TEXT | Product Name | | salary | REAL | Price in EGP | | stock | INTEGER | Available quantity | | image | TEXT | Path to image file |

Table: Cart | Field | Type | Description | | :--- | :--- | :--- | | id | INTEGER PK | Unique Cart Item ID | | user_id | INTEGER | FK ref Users | | product_id | INTEGER | FK ref Products | | quantity | INTEGER | Quantity to buy

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

5. Behavioral Model and Description

5.2. Description for Software Behavior

- **Startup:** When the application starts, it initializes the database (init_db.py) and ensures the default Admin user exists.
- **Navigation:** Users start at the Home page. If they try to access Admin pages without the 'admin' role in session_state, the system blocks access and shows a warning.
- **Transaction:** When a payment is successful, the behavioral logic ensures that the Stock is reduced instantly to prevent overselling, and the Cart is emptied.

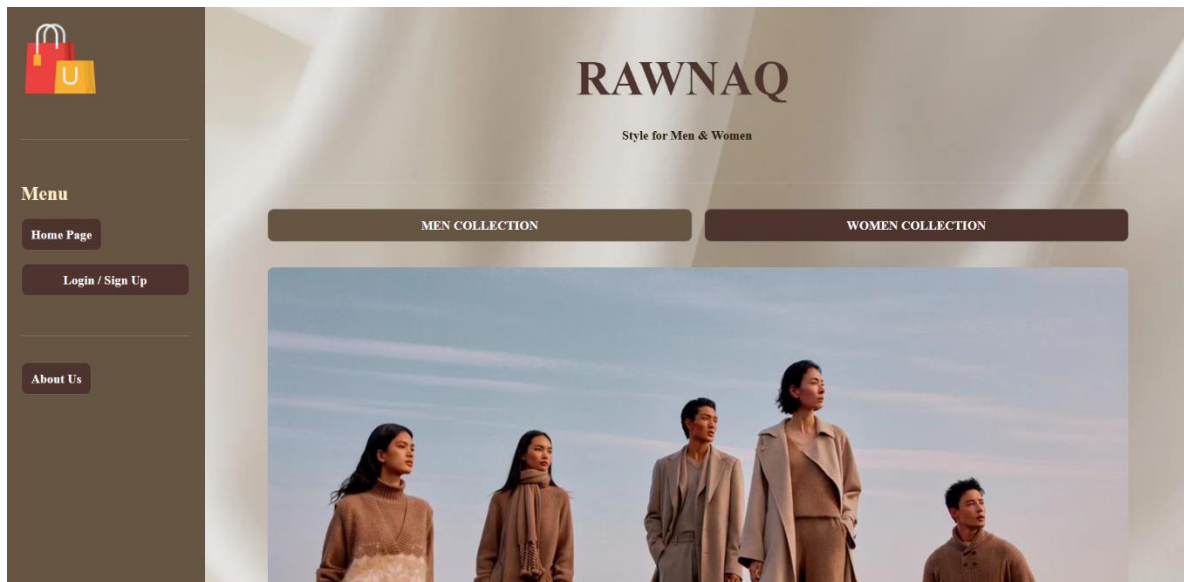


Figure 3. Picture of Main Page

Users can display the login page via “Login” button. The users can enter the system by entering username and password on this page.

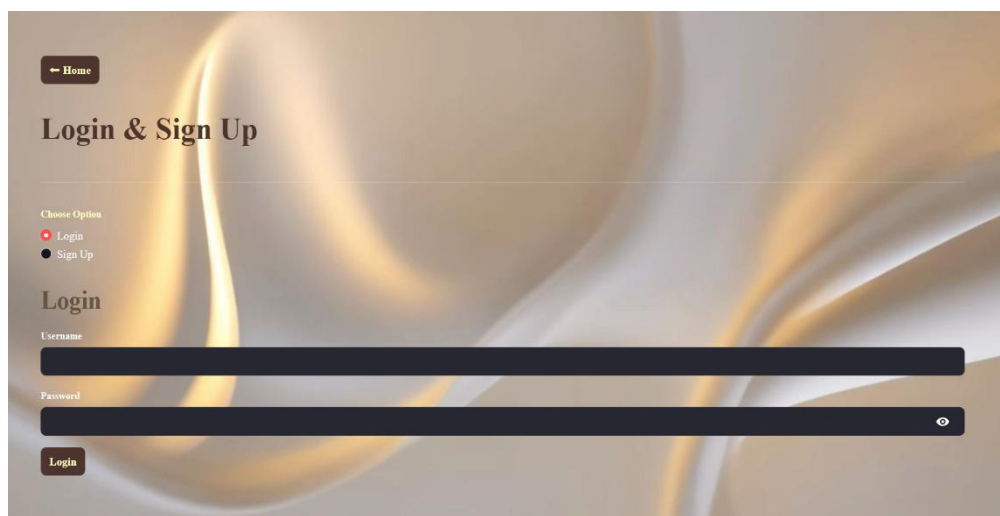


Figure 4. Picture of Log In Page

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

If they do not have an account, they can register the system by clicking “Sign Up” button. In this page, users have to enter their username, password.

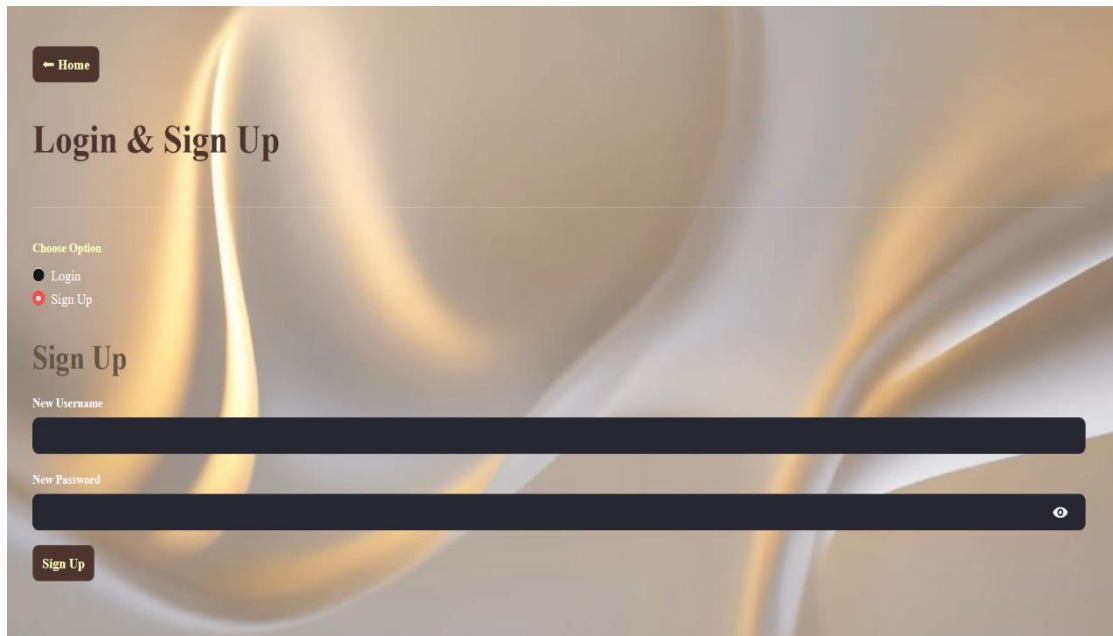
The image shows a web page titled "Login & Sign Up". At the top left, there is a "Home" button. Below the title, there is a "Choose Option" section with two radio buttons: "Login" (selected) and "Sign Up". The "Sign Up" option is highlighted with a red dot. Below this, there is a "Sign Up" section with two input fields: "New Username" and "New Password". The "New Password" field has a toggle icon (an eye) to the right of it. At the bottom of the form, there is a "Sign Up" button. The background of the page is a blurred image of a person's face.

Figure 5. Visualization of Sign Up Page

When they enter the system, they see their profile information such as name, password, and they can change them, and log out

The image shows a web page titled "Update Name". It has two main sections: "Update Name" and "Security". The "Update Name" section has a "New Name" input field with the text "lynah" and an "Update Name" button. The "Security" section has a "Current Password" input field with a toggle icon (an eye) to the right of it. Below this, there is a "Change Password" section with three input fields: "Current Password", "New Password", and "Confirm New Password". Each of these three input fields has a toggle icon (an eye) to the right of it. The background of the page is a blurred image of a person's face.

Figure 6. Visualization of Profile (user) Page

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

User can Chose products and add it to the cart and filter the products by price and name.

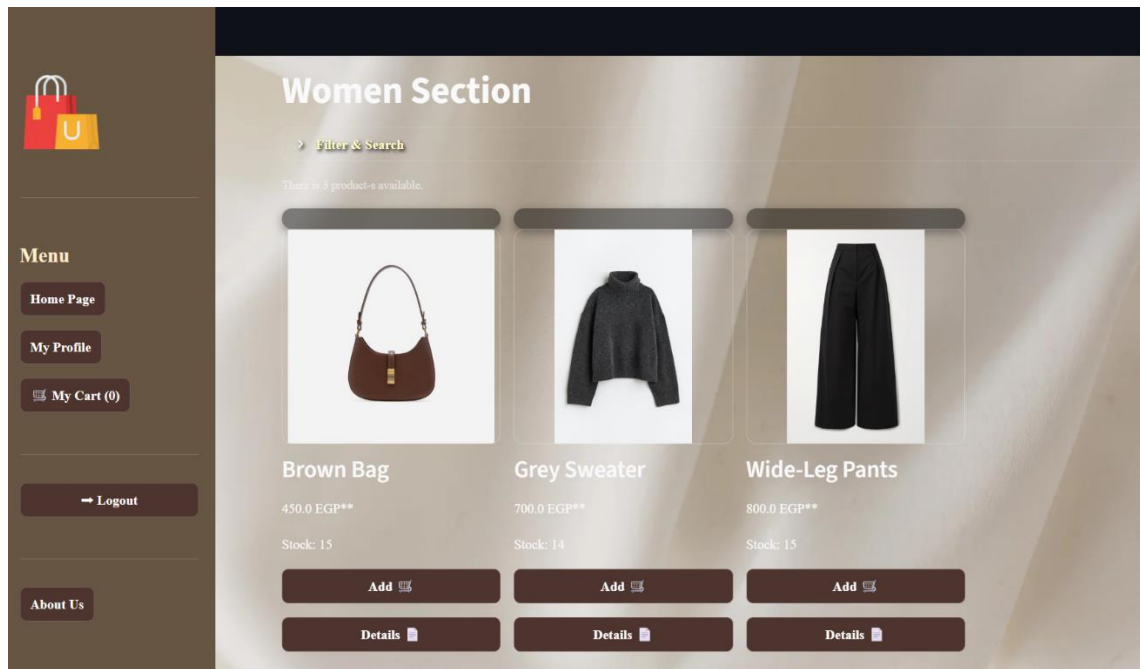


Figure 7. Visualization of Products & Filtering Page

Then if the user click on product, the page of details will shown.

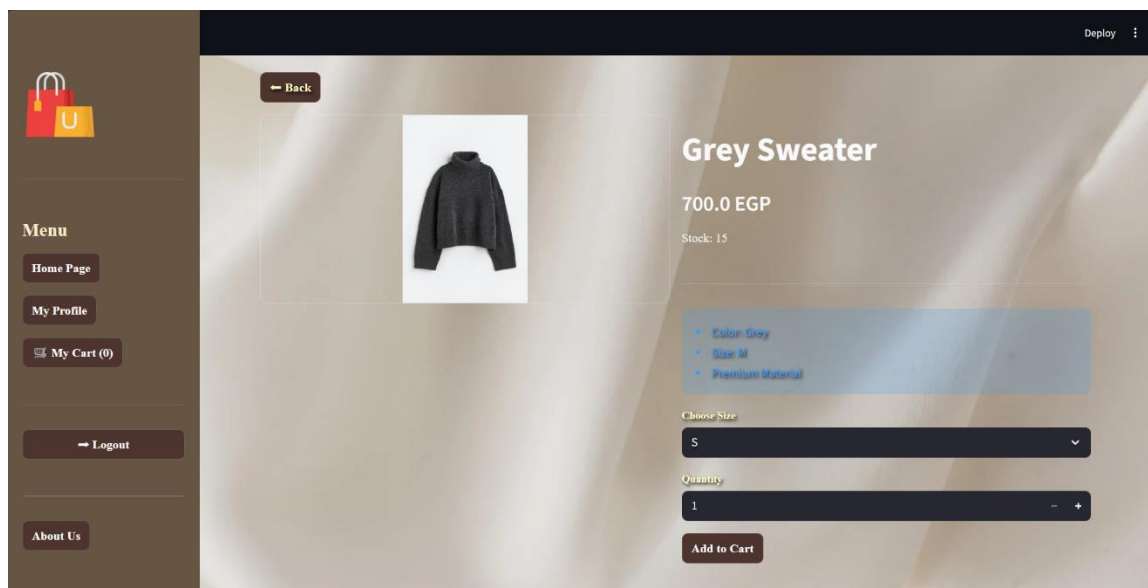


Figure 8. Visualization of Details Page

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

Moreover, users can see his cart and remove products from it and move to checkout.

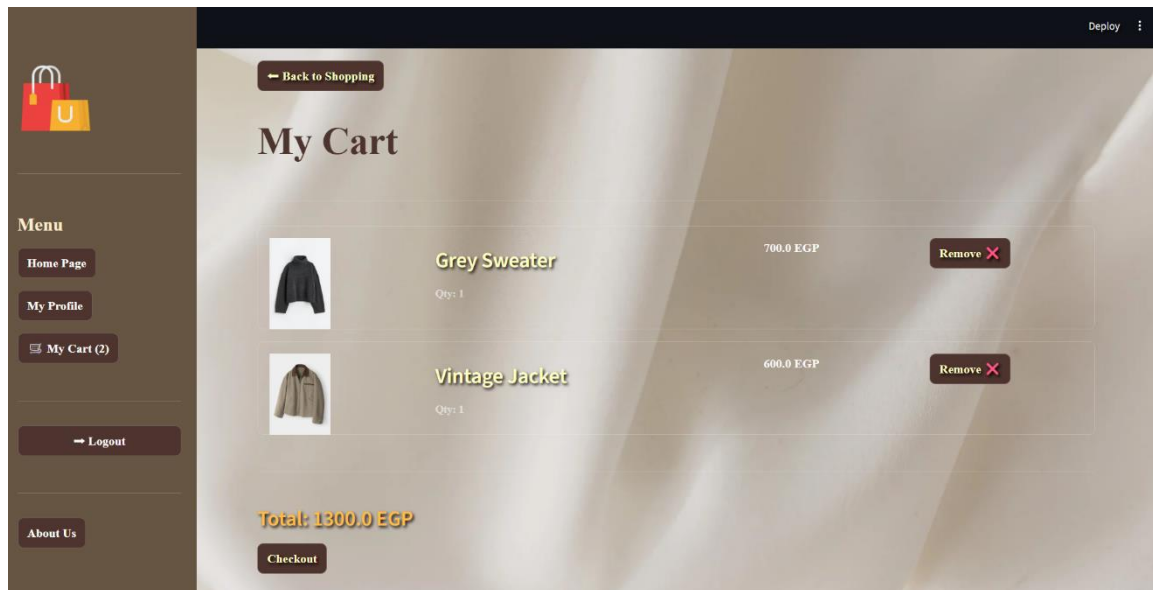


Figure 9. Visualization of Cart Page

If user go to the checkout he will go to the Payment page.



Figure 10. Visualization of Payment Page

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

When users click About us the details of our Brand will be shown.

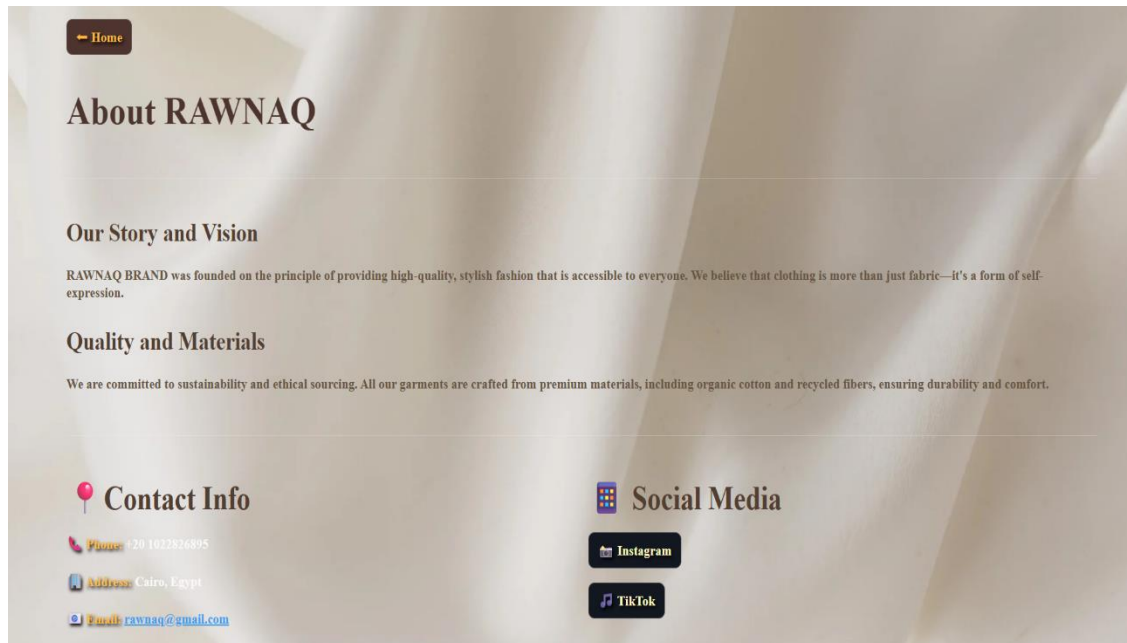


Figure 11. Visualization of Events Page

If user log in as **Admin** he will move to the Admin Dashboard page.

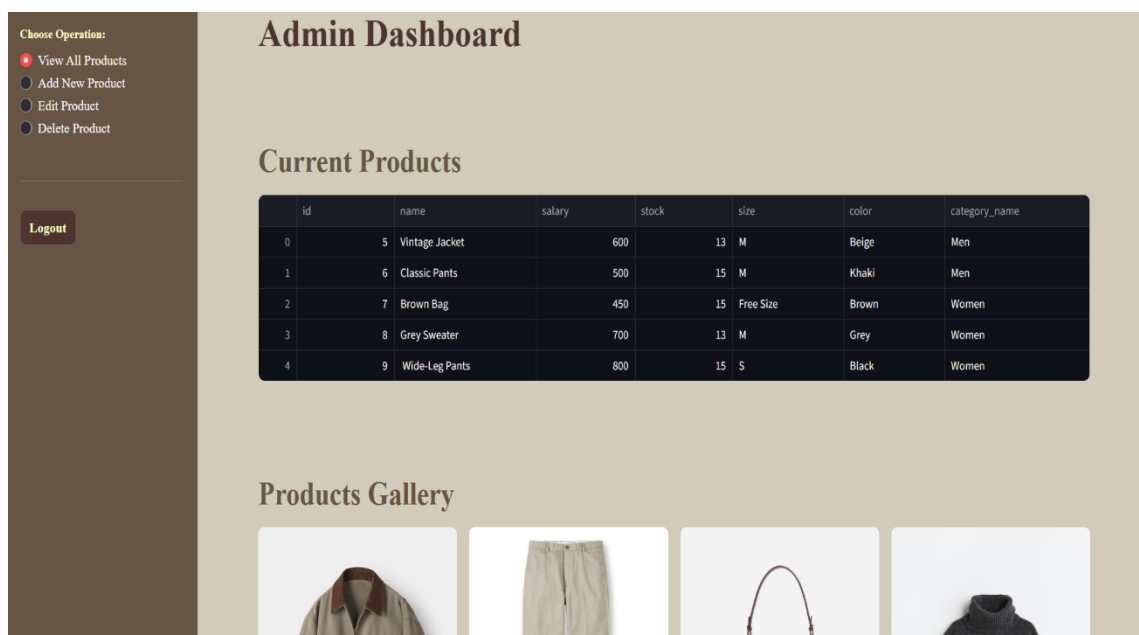


Figure 12. Visualization of Admin Dashboard Page

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

When click add product the Add product page will open, and he can add.

The screenshot shows the 'Admin Dashboard' with a sidebar on the left containing a 'Choose Operation:' menu with options: View All Products, Add New Product (highlighted with a red dot), Edit Product, and Delete Product. Below the menu is a 'Logout' button. The main content area is titled 'Add New Product' and contains several input fields: 'Product Name' (text input), 'Size' (dropdown menu with 'S' selected), 'Category' (dropdown menu with 'Men' selected), 'Color' (text input with 'Black'), 'Price (EGP)' (text input with '1.00' and minus/plus buttons), and 'Stock Quantity' (text input with '1' and minus/plus buttons). Below these fields is a 'Product Image' section with an 'Upload Image' label, a cloud icon, the text 'Drag and drop file here', 'Limit 200MB per file • PNG, JPG, JPEG', and a 'Browse files' button. At the bottom of the form is a 'Save Product' button.

Figure 13. Visualization of Add new product Page

When Admin click Edit product he can edit the details of each product.

The screenshot shows the 'Admin Dashboard' with the same sidebar as Figure 13. The main content area is titled 'Edit Product Details' and features a 'Select Product' dropdown menu with 'Vintage Jacket (ID: 5)' selected. Below this are input fields for 'Name' (text input with 'Vintage Jacket'), 'Price' (text input with '600.00' and minus/plus buttons), and 'Stock' (text input with '15' and minus/plus buttons). At the bottom of the form is an 'Update Product' button.

Figure 14. Visualization of Edit product Page

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

When Admin click Delete product he can delete a specific product.

Choose Operation:

- View All Products
- Add New Product
- Edit Product
- Delete Product

Logout

Admin Dashboard

Edit Product Details

Select Product

Vintage Jacket (ID: 5)

Name

Vintage Jacket

Price

600.00

Stock

15

Update Product

Figure 15. Visualization of Delete product Page

5. Planning

5.1. Team Structure

The structure of team is arranged as work-based. Everyone may have different task. However, the arrangement of team works is assigned to every person. The arrangement is listed below:

- **Boys Staff:**
 - Ahmed Helmy
 - Ammar Yasser
 - Abdelrhman Ashraf
 - Ahmed Saber
 - Ahmed Mohammad
- **Girls Staff:**
 - Rokaya Alaa
 - Mariam Osama
 - Maya Seraj
 - Lynah Adel
 - Walaa Magdy

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

Responsibilities Distribution:

- **Backend & Database:** Responsible for team5.py and init_db.py.
- **Frontend (Streamlit):** Responsible for Homepage.py and Admin_Dashboard.py.
- **Payment & Integration:** Responsible for Payment.py.
- **Testing & Documentation:** Responsible for testing scenarios and SRS creation.

5.2. Estimation

The estimation of basic schedule is listed below,

- **Database Design:** Completed.
- **Core Functionality (Auth, Cart):** Completed.
- **UI Implementation:** Completed.
- **Testing:** Completed (Verified via team5.py test block).

5.3. Process Model

The project followed an **Agile-like** approach where the Database was built first, followed by core CRUD functions, and finally the User Interface was layered on top using Streamlit for rapid prototyping and deployment.

6. Conclusion

The **RAWNAQ** project successfully implements a functional E-Commerce platform meeting the requirements of browsing, purchasing, and administration. The system utilizes a robust SQLite database and a responsive Streamlit interface. This SRS document has detailed the specific functional and non-functional requirements that guided the development process, ensuring all stakeholder needs (Admins and Customers) are met.

7. Supporting Information

At the beginning of the document, table of contents and list of figures are included.

SYSTEM REQUIREMENT SPECIFICATION OF SUCH

SYSTEM REQUIREMENT SPECIFICATION OF SUCH